Predicting Airbnb Listing Price Across Different Cities

UE24CS352A Machine Learning Mini Project
Sakaray Vishal (PES1UG24AM241) and Sreedharala Praneetha (PES1UG24AM281)
Code: https://github.com/vxs01/Airbnb-Price-Prediction-MLMini

1. Problem statement

Airbnb hosts mostly set their own prices by guessing. That is hard for new hosts: if the price is too high nobody books, and if it is too low they lose money. In this project we reproduce the CS229 report "Predicting Airbnb Listing Price Across Different Cities" (Luo, Zhou and Zhou, 2019).

The task is to predict the price per night of a listing from its details (size, location, reviews, amenities, description and so on). We do this in two parts:

- **Single city:** train and test a model inside one city (Berlin, Paris or NYC).
- **Cross city:** train a model on NYC, on Paris, and on NYC and Paris together, and then test it on Berlin, a city it has never seen. This shows whether a pricing model can carry over to a new city.

2. Dataset

We used the Inside Airbnb listing files for Berlin, Paris and NYC. Each row is one listing and has about 80 to 90 columns (price, number of bedrooms, location, review scores, amenities, text description and more).

City	Rows in file	Rows after cleaning	Scrape year
Berlin	14,274	9,133	2025
Paris	38,075	30,971	2026
NYC	30,259	19,148	2026

Cleaning steps, the same for all three cities:

- Prices were converted from text like "$85.00" to numbers, and listings without a price were removed.
- Like the report, we only kept listings priced at $500 per night or less. A few listings cost thousands per night and would otherwise distort the models.
- We predict the **log of the price** instead of the price itself. This also reduces the effect of very expensive listings and helps the models.

Note that the Berlin file is from an earlier scrape (2025) than the Paris and NYC files (2026), so the data is not from exactly the same period.

3. Approach

3.1 Single-city models

For each city we turned every listing into a row of numbers (features):

- **Numbers** such as accommodates, bedrooms, beds, latitude, longitude and review scores. Missing values were filled with the median.
- **Dates** (first review, last review) converted to numbers.
- **Categories** such as room type, property type and neighbourhood, converted to 0/1 columns (one-hot encoding).

- **Amenities** (wifi, kitchen, washer, ...) as one 0/1 column per amenity.
- **Text** (name, description and host about, plus neighbourhood overview for Berlin) converted with TF-IDF and then reduced to 50 values per text field using SVD.

All features were standardized using the training data only. Each city was split 70% train, 20% dev and 10% test, and five models were trained:

- **Linear Regression:** finds the best straight-line relationship between features and price.
- **KNN (k = 5):** predicts the average price of the 5 most similar listings.
- **Random Forest (100 trees):** averages the predictions of many decision trees.
- **XGBoost (100 trees, depth 6, learning rate 0.1):** builds trees one after another, each one correcting the mistakes of the previous ones.
- **Neural network:** hidden layers of 128, 512 and 64 units with dropout 0.2, trained with Adam (learning rate 1e-4) for 100 epochs.

3.2 Cross-city models

The single-city feature sets have different sizes (Berlin has 2,858 features, Paris 5,448 and NYC 7,182), so a model trained on one cannot be used on another. For the cross-city part we therefore built one common feature set that works for every city:

- 25 numeric columns, 7 category columns (room type, property type, host flags, availability, instant bookable) and the amenities that are common enough.
- As in the report, the text features were left out, because the text features come out differently in each city and cannot be matched up.

To make sure the test is fair, everything the features depend on (medians, category lists, amenity list, scaling) is learned from the training city or cities only. Berlin is then converted with those same settings, and Berlin is never used for training. We ran three experiments: NYC to Berlin, Paris to Berlin, and NYC plus Paris to Berlin.

The model is a small neural network following the report: two hidden layers (64 and 32 units), ReLU, mean squared error loss, SGD with Nesterov momentum (0.9, learning rate 0.005), L2 regularization 0.005, and early stopping using 10% of the training data as a check set.

4. Implementation

All the code is in one Google Colab notebook (notebooks/airbnb_price_prediction.ipynb) in the GitHub repository. It uses pandas, scikit-learn, XGBoost, PyTorch and Keras. The notebook has one section for each city with the baseline models, and a last section called "Cross city Prediction". That section has three main parts: a function that builds the common features for any city, a function that trains the neural network with early stopping, and a loop that runs the three experiments and saves the results table to results/cross_city_results.csv. The README in the repository explains where to put the data files and how to run the notebook.

We report two measures on the log price. **MSE** (mean squared error) is the average squared mistake, so lower is better. **R2** shows how much of the variation in price the model explains: 1 is perfect, 0 is no better than always guessing the average price, and a value below 0 is worse than that.

5. Results

Table 1: single-city test results (MSE / R2)

Model	Berlin	Paris	NYC
Linear Regression	0.1448 / 0.578	0.0886 / 0.620	0.1218 / 0.713
KNN	0.2408 / 0.298	0.1602 / 0.313	0.2478 / 0.416
Random Forest	0.1120 / 0.673	0.0823 / 0.647	0.0851 / 0.799
XGBoost	**0.0989 / 0.712**	**0.0647 / 0.723**	**0.0800 / 0.811**
Neural Network	0.3036 / 0.115	0.1215 / 0.479	0.4662 / -0.100

XGBoost (in bold) was the best model in all three cities. Random Forest was close behind and KNN was the weakest. Our neural network did much worse than the tree models: R2 of 0.11 for Berlin, 0.48 for Paris and -0.10 for NYC.

Table 2: cross-city transfer, neural network, tested on all of Berlin

Trained on	Features	Train R2	Berlin MSE	Berlin R2
NYC	258	0.869	2850.29	-8243.98
Paris	234	0.733	30.87	-88.30
NYC + Paris	**271**	**0.744**	**0.315**	**0.088**

The models trained on one city only fit their own city well (train R2 of 0.87 and 0.73) but failed on Berlin, with R2 far below zero. Berlin has feature values and price levels that these models never saw, so their predictions were far off. The model trained on NYC and Paris together did much better on Berlin (MSE 0.315, R2 0.088).

6. Conclusions

- Inside one city, XGBoost is the best choice. It gave R2 of 0.712 for Berlin, 0.723 for Paris and 0.811 for NYC.
- A model trained on a single city does not transfer to a new city. The NYC-only and Paris-only models had a very negative R2 on Berlin.
- Training on several cities together gives a much better result on a new city (R2 0.088 against -8244 and -88). This agrees with the report: more varied training cities give a more general model.

Limitations. Our combined-city R2 of 0.088 is a lot lower than the 0.514 in the report. Likely reasons: we used different and newer data files than the report; prices differ a lot between cities (median price about $103 in Berlin, $203 in Paris and $156 in NYC); the Berlin data is from a different year than Paris and NYC; and we did not tune the neural network. Also, the single-city results change slightly from run to run (the text SVD and the neural networks start from random values), so a re-run can differ in the second or third decimal. The cross-city results are fixed by a random seed.

Future work. Adjust prices for each city before training, add more training cities, and tune the cross-city model.

Reference: Y. Luo, X. Zhou and Y. Zhou, "Predicting Airbnb Listing Price Across Different Cities", CS229 course project, Stanford, December 2019.